

1 系统模型

1.1 系统架构

本研究考虑一个由终端设备（如医疗物联网设备）、边缘服务器和云服务器组成的三层计算架构。系统由以下几个集合组成：

- 设备集合： $\mathcal{D} = \{1, 2, \dots, D\}$ ，表示 D 个终端设备
- 边缘服务器集合： $\mathcal{E} = \{1, 2, \dots, E\}$ ，表示 E 个边缘服务器
- 云服务器集合： $\mathcal{C} = \{1, 2, \dots, C\}$ ，表示 C 个云服务器
- 任务集合： $\mathcal{T} = \{1, 2, \dots, T\}$ ，表示 T 个计算任务

1.2 任务模型

每个计算任务 $i \in \mathcal{T}$ 由以下基本参数描述：

$$D_i : \text{任务 } i \text{ 的数据大小 (单位: 比特)} \quad (1)$$

$$C_i : \text{任务 } i \text{ 的计算复杂度 (单位: CPU 周期/比特)} \quad (2)$$

$$\lambda_i : \text{任务 } i \text{ 的到达率 (单位: 任务/秒)} \quad (3)$$

$$\rho_i : \text{任务 } i \text{ 的优先级因子, 表示其重要性} \quad (4)$$

$$T_i^{\max} : \text{任务 } i \text{ 的最大可接受延迟} \quad (5)$$

1.3 决策变量

主要决策变量包括任务卸载决策和资源分配决策：

$$x_{i,j}^d \in \{0, 1\} : \text{若任务 } i \text{ 在设备 } j \text{ 本地执行则为 1, 否则为 0} \quad (6)$$

$$x_{i,j}^e \in \{0, 1\} : \text{若任务 } i \text{ 卸载到边缘服务器 } j \text{ 执行则为 1, 否则为 0} \quad (7)$$

$$x_{i,j}^c \in \{0, 1\} : \text{若任务 } i \text{ 卸载到云服务器 } j \text{ 执行则为 1, 否则为 0} \quad (8)$$

$$f_{i,j}^d : \text{分配给设备 } j \text{ 上任务 } i \text{ 的计算资源 (CPU 频率, 单位: Hz)} \quad (9)$$

$$f_{i,j}^e : \text{分配给边缘服务器 } j \text{ 上任务 } i \text{ 的计算资源} \quad (10)$$

$$f_{i,j}^c : \text{分配给云服务器 } j \text{ 上任务 } i \text{ 的计算资源} \quad (11)$$

2 第一部分：高能效任务卸载和资源分配模型

在第一部分中，我们专注于优化系统能耗、处理延迟和计算资源分配，而不考虑信息年龄 (AoI)。

2.1 延迟模型

任务 i 的总延迟 T_i 由以下几部分组成：

2.1.1 计算延迟

$$T_{i,j}^{d,comp} = \frac{D_i \cdot C_i}{f_{i,j}^d} \quad (\text{设备本地计算延迟}) \quad (12)$$

$$T_{i,j}^{e,comp} = \frac{D_i \cdot C_i}{f_{i,j}^e} \quad (\text{边缘服务器计算延迟}) \quad (13)$$

$$T_{i,j}^{c,comp} = \frac{D_i \cdot C_i}{f_{i,j}^c} \quad (\text{云服务器计算延迟}) \quad (14)$$

2.1.2 传输延迟

$$T_{i,j,k}^{d,e,trans} = \frac{D_i}{R_{j,k}^{d,e}} \quad (\text{设备 } j \text{ 到边缘服务器 } k \text{ 的传输延迟}) \quad (15)$$

$$T_{i,j,k}^{e,c,trans} = \frac{D_i}{R_{j,k}^{e,c}} \quad (\text{边缘服务器 } j \text{ 到云服务器 } k \text{ 的传输延迟}) \quad (16)$$

其中 $R_{j,k}^{d,e}$ 是设备 j 到边缘服务器 k 的传输速率， $R_{j,k}^{e,c}$ 是边缘服务器 j 到云服务器 k 的传输速率。

2.1.3 排队延迟

考虑到系统中可能存在多个任务竞争资源，引入排队延迟：

$$T_{i,j}^{d,queue} = \text{设备 } j \text{ 上任务 } i \text{ 的排队延迟} \quad (17)$$

$$T_{i,j}^{e,queue} = \text{边缘服务器 } j \text{ 上任务 } i \text{ 的排队延迟} \quad (18)$$

$$T_{i,j}^{c,queue} = \text{云服务器 } j \text{ 上任务 } i \text{ 的排队延迟} \quad (19)$$

排队延迟可以使用排队论模型进行估计，这里使用 M/M/1 队列模型：

$$T_{i,j}^{d,queue} = \frac{\lambda_j^d \cdot (D_i \cdot C_i)^2}{f_{i,j}^d \cdot (f_{i,j}^d - \lambda_j^d \cdot D_i \cdot C_i)} \quad (20)$$

$$T_{i,j}^{e,queue} = \frac{\lambda_j^e \cdot (D_i \cdot C_i)^2}{f_{i,j}^e \cdot (f_{i,j}^e - \lambda_j^e \cdot D_i \cdot C_i)} \quad (21)$$

$$T_{i,j}^{c,queue} = \frac{\lambda_j^c \cdot (D_i \cdot C_i)^2}{f_{i,j}^c \cdot (f_{i,j}^c - \lambda_j^c \cdot D_i \cdot C_i)} \quad (22)$$

其中 λ_j^d , λ_j^e , λ_j^c 分别是设备 j , 边缘服务器 j 和云服务器 j 上的任务到达率。

此公式基于排队论中的 Pollaczek-Khinchine 公式的简化版，适用于 M/M/1 系统。M/M/1 表示一个具有泊松到达过程、指数服务时间分布和单一服务器的排队系统。以设备上的排队延迟为例，推导过程如下：

1. 平均服务时间 $S = \frac{D_i \cdot C_i}{f_{i,j}^d}$
2. 服务强度 (利用率) $\rho = \lambda_j^d \cdot S = \frac{\lambda_j^d \cdot D_i \cdot C_i}{f_{i,j}^d}$
3. M/M/1 系统的平均等待时间 $W_q = \frac{\rho \cdot S}{1 - \rho} = \frac{\frac{\lambda_j^d \cdot D_i \cdot C_i}{f_{i,j}^d} \cdot \frac{D_i \cdot C_i}{f_{i,j}^d}}{1 - \frac{\lambda_j^d \cdot D_i \cdot C_i}{f_{i,j}^d}}$
4. 简化得到最终公式 $T_{i,j}^{d,queue} = \frac{\lambda_j^d \cdot (D_i \cdot C_i)^2}{f_{i,j}^d \cdot (f_{i,j}^d - \lambda_j^d \cdot D_i \cdot C_i)}$

上述公式的稳定性条件：

$$f_{i,j}^d > \lambda_j^d \cdot D_i \cdot C_i \quad (23)$$

$$f_{i,j}^e > \lambda_j^e \cdot D_i \cdot C_i \quad (24)$$

$$f_{i,j}^c > \lambda_j^c \cdot D_i \cdot C_i \quad (25)$$

这些条件确保服务能力大于到达工作负载，否则队列将无限增长，系统将变得不稳定。在实际应用中，这些条件必须作为资源分配的约束条件。

2.1.4 总延迟

任务 i 的总延迟可以表示为：

$$\begin{aligned} T_i = & \sum_{j \in \mathcal{D}} x_{i,j}^d \cdot (T_{i,j}^{d,comp} + T_{i,j}^{d,queue}) + \sum_{j \in \mathcal{E}} \sum_{k \in \mathcal{D}} x_{i,j}^e \cdot (T_{i,k,j}^{d,e,trans} + T_{i,j}^{e,comp} + T_{i,j}^{e,queue}) \\ & + \sum_{j \in \mathcal{C}} \sum_{k \in \mathcal{E}} \sum_{l \in \mathcal{D}} x_{i,j}^c \cdot (T_{i,l,k}^{d,e,trans} + T_{i,k,j}^{e,c,trans} + T_{i,j}^{c,comp} + T_{i,j}^{c,queue}) \end{aligned} \quad (26)$$

2.2 能耗模型

2.2.1 计算能耗

$$E_{i,j}^{d,comp} = \kappa \cdot (f_{i,j}^d)^2 \cdot D_i \cdot C_i \quad (\text{设备本地计算能耗}) \quad (27)$$

$$E_{i,j}^{e,comp} = \kappa_e \cdot (f_{i,j}^e)^2 \cdot D_i \cdot C_i \quad (\text{边缘服务器计算能耗}) \quad (28)$$

$$E_{i,j}^{c,comp} = \kappa_c \cdot (f_{i,j}^c)^2 \cdot D_i \cdot C_i \quad (\text{云服务器计算能耗}) \quad (29)$$

其中 $\kappa, \kappa_e, \kappa_c$ 是能耗系数。

2.2.2 传输能耗

$$E_{i,j,k}^{d,e,trans} = P_j^{trans} \cdot \frac{D_i}{R_{j,k}^{d,e}} \quad (\text{设备 } j \text{ 到边缘服务器 } k \text{ 的传输能耗}) \quad (30)$$

$$E_{i,j,k}^{e,c,trans} = P_j^{e,trans} \cdot \frac{D_i}{R_{j,k}^{e,c}} \quad (\text{边缘服务器 } j \text{ 到云服务器 } k \text{ 的传输能耗}) \quad (31)$$

其中 P_j^{trans} 和 $P_j^{e,trans}$ 分别是设备 j 和边缘服务器 j 的传输功率。

2.2.3 总能耗

任务 i 的总能耗可以表示为：

$$\begin{aligned} E_i = & \sum_{j \in \mathcal{D}} x_{i,j}^d \cdot E_{i,j}^{d,comp} + \sum_{j \in \mathcal{E}} \sum_{k \in \mathcal{D}} x_{i,j}^e \cdot (E_{i,k,j}^{d,e,trans} + E_{i,j}^{e,comp}) \\ & + \sum_{j \in \mathcal{C}} \sum_{k \in \mathcal{E}} \sum_{l \in \mathcal{D}} x_{i,j}^c \cdot (E_{i,l,k}^{d,e,trans} + E_{i,k,j}^{e,c,trans} + E_{i,j}^{c,comp}) \end{aligned} \quad (32)$$

2.3 可靠性模型

在医疗场景中，系统的可靠性至关重要。引入任务执行失败概率：

$$p_{i,j}^{d,fail} : \text{任务 } i \text{ 在设备 } j \text{ 上执行失败的概率} \quad (33)$$

$$p_{i,j}^{e,fail} : \text{任务 } i \text{ 在边缘服务器 } j \text{ 上执行失败的概率} \quad (34)$$

$$p_{i,j}^{c,fail} : \text{任务 } i \text{ 在云服务器 } j \text{ 上执行失败的概率} \quad (35)$$

任务 i 的成功执行概率可以表示为：

$$p_i^{success} = \sum_{j \in \mathcal{D}} x_{i,j}^d \cdot (1 - p_{i,j}^{d,fail}) + \sum_{j \in \mathcal{E}} x_{i,j}^e \cdot (1 - p_{i,j}^{e,fail}) + \sum_{j \in \mathcal{C}} x_{i,j}^c \cdot (1 - p_{i,j}^{c,fail}) \quad (36)$$

2.4 第一部分优化问题

2.4.1 目标函数

第一部分的目标函数主要考虑能耗和延迟的双目标优化：

$$\min F_1 = \sum_{i \in \mathcal{T}} \left(w_i^E \cdot \frac{E_i}{E_{max}} + w_i^T \cdot \frac{T_i}{T_{max}} \right) \quad (37)$$

其中 w_i^E , w_i^T 是权重系数，分别反映了能耗和延迟的相对重要性。 E_{max} , T_{max} 是归一化因子。

2.4.2 约束条件

任务分配约束 每个任务必须且只能分配给一个执行位置：

$$\sum_{j \in \mathcal{D}} x_{i,j}^d + \sum_{j \in \mathcal{E}} x_{i,j}^e + \sum_{j \in \mathcal{C}} x_{i,j}^c = 1, \quad \forall i \in \mathcal{T} \quad (38)$$

资源容量约束 计算资源约束：

$$\sum_{i \in \mathcal{T}} x_{i,j}^d \cdot f_{i,j}^d \leq F_j^d, \quad \forall j \in \mathcal{D} \quad (39)$$

$$\sum_{i \in \mathcal{T}} x_{i,j}^e \cdot f_{i,j}^e \leq F_j^e, \quad \forall j \in \mathcal{E} \quad (40)$$

$$\sum_{i \in \mathcal{T}} x_{i,j}^c \cdot f_{i,j}^c \leq F_j^c, \quad \forall j \in \mathcal{C} \quad (41)$$

其中 F_j^d , F_j^e , F_j^c 分别是设备 j , 边缘服务器 j 和云服务器 j 的最大计算资源。

带宽约束：

$$\sum_{i \in \mathcal{T}} \sum_{j \in \mathcal{E}} x_{i,j}^e \cdot \frac{D_i}{R_{k,j}^{d,e}} \leq B_{k,j}^{d,e}, \quad \forall k \in \mathcal{D}, j \in \mathcal{E} \quad (42)$$

$$\sum_{i \in \mathcal{T}} \sum_{j \in \mathcal{C}} x_{i,j}^c \cdot \frac{D_i}{R_{k,j}^{e,c}} \leq B_{k,j}^{e,c}, \quad \forall k \in \mathcal{E}, j \in \mathcal{C} \quad (43)$$

其中 $B_{k,j}^{d,e}$ 和 $B_{k,j}^{e,c}$ 分别是设备 k 到边缘服务器 j 和边缘服务器 k 到云服务器 j 的带宽上限。

性能约束 延迟约束：

$$T_i \leq T_i^{\max}, \quad \forall i \in \mathcal{T} \quad (44)$$

可靠性约束：

$$p_i^{\text{success}} \geq p_i^{\min}, \quad \forall i \in \mathcal{T} \quad (45)$$

其中 $p_i^{\min}$ 是任务 i 的最小可接受成功概率。

能耗约束：

$$\sum_{i \in \mathcal{T}} \sum_{j \in \mathcal{D}} x_{i,j}^d \cdot E_{i,j}^{d,comp} \leq E_j^{\max}, \quad \forall j \in \mathcal{D} \quad (46)$$

其中 $E_j^{\max}$ 是设备 j 的最大能耗限制，这对于电池供电的医疗设备尤为重要。

3 第二部分：基于信息年龄优化的任务卸载和资源分配模型

在第二部分中，我们在第一部分的基础上引入信息年龄 (AoI) 概念，研究如何优化医疗信息的时效性。

3.1 任务模型扩展

为了考虑 AoI，我们扩展任务模型，增加以下参数：

$$\Delta_i : \text{任务 } i \text{ 的更新间隔 (单位: 秒)} \quad (47)$$

$$A_i^{\max} : \text{任务 } i \text{ 的最大可接受信息年龄 (AoI)} \quad (48)$$

3.2 信息年龄 (AoI) 模型

3.2.1 AoI 定义

在时间 t ，任务 i 的信息年龄定义为：

$$A_i(t) = t - U_i(t) \quad (49)$$

其中 $U_i(t)$ 是任务 i 的最后一次成功更新的时间戳。

3.2.2 平均 AoI

在时间间隔 $[0, T]$ 内，任务 i 的平均 AoI 为：

$$\bar{A}_i = \frac{1}{T} \int_0^T A_i(t) dt \quad (50)$$

对于周期性任务，假设更新间隔为固定值 Δ_i ，则平均 AoI 可以简化为：

$$\bar{A}_i = \frac{\Delta_i}{2} + T_i \quad (51)$$

其中 T_i 是任务 i 的总延迟（从第一部分计算得到）。

3.2.3 峰值 AoI

峰值 AoI 是指两次连续更新之间的最大 AoI 值：

$$A_i^{peak} = \Delta_i + T_i \quad (52)$$

3.3 AoI 优化策略

3.3.1 更新间隔优化

对于动态任务，可以优化更新间隔 Δ_i 以平衡 AoI 和系统资源消耗：

$$\Delta_i^{opt} = \arg \min_{\Delta_i} \left(\alpha \cdot \bar{A}_i + \beta \cdot \frac{1}{\Delta_i} \cdot E_i \right) \quad (53)$$

其中 α 和 β 是权重系数， $\frac{1}{\Delta_i} \cdot E_i$ 表示单位时间内的能耗。

3.3.2 优先级调度

基于 AoI 的优先级调度策略：

$$\pi_i(t) = \gamma \cdot A_i(t) + (1 - \gamma) \cdot \rho_i \quad (54)$$

其中 $\pi_i(t)$ 是任务 i 在时间 t 的调度优先级， γ 是权重系数， ρ_i 是任务 i 的固有优先级。

3.4 第二部分优化问题

3.4.1 目标函数

第二部分的目标函数考虑了能耗、延迟和 AoI 的三目标优化：

$$\min F_2 = \sum_{i \in \mathcal{T}} \left(w_i^E \cdot \frac{E_i}{E_{max}} + w_i^T \cdot \frac{T_i}{T_{max}} + w_i^A \cdot \frac{\bar{A}_i}{A_{max}} \right) \quad (55)$$

其中 w_i^E, w_i^T, w_i^A 是权重系数, $E_{max}, T_{max}, A_{max}$ 是归一化因子。

3.4.2 约束条件

第二部分包含第一部分的所有约束条件, 并额外增加 AoI 约束：

$$\bar{A}_i \leq A_i^{\max}, \quad \forall i \in \mathcal{T} \quad (56)$$

此外, 为了优化更新频率, 还可以增加以下约束：

$$\Delta_i^{\min} \leq \Delta_i \leq \Delta_i^{\max}, \quad \forall i \in \mathcal{T} \quad (57)$$

其中 $\Delta_i^{\min}$ 和 $\Delta_i^{\max}$ 分别是任务 i 的最小和最大更新间隔。

4 启发式算法求解

由于上述优化问题是混合整数非线性规划问题 (MINLP), 属于 NP 难问题, 因此需要使用启发式算法进行求解。

4.1 第一部分求解算法

对于第一部分 (不考虑 AoI 的高能效任务卸载和资源分配), 可以采用以下算法：

- 灰狼优化算法 (GWO)
- 粒子群优化 (PSO)
- 遗传算法 (GA)
- 模拟退火算法 (SA)

4.2 第二部分求解算法

对于第二部分（考虑 AoI 的任务卸载和资源分配），由于涉及三个目标函数，可以采用多目标优化算法：

- 非支配排序遗传算法-II (NSGA-II)
- 多目标灰狼优化算法 (MOGWO)
- 多目标粒子群优化算法 (MOPSO)
- 多目标进化算法 (MOEA/D)

4.3 算法编码

在使用启发式算法求解时，需要对决策变量进行编码。编码是将优化问题中的决策变量转换为启发式算法可以处理的形式，是连接数学模型和算法实现的桥梁。

对于第一部分问题，目前的编码方式是：

$$S_i = [loc_i, f_i], \quad \forall i \in \mathcal{T} \quad (58)$$

对于第二部分问题，编码需要包含更新间隔：

$$S_i = [loc_i, f_i, \Delta_i], \quad \forall i \in \mathcal{T} \quad (59)$$

其中 loc_i 表示任务 i 的执行位置，可以编码为：

- $loc_i = 0, 1, \dots, D - 1$: 在设备上执行
- $loc_i = D, D + 1, \dots, D + E - 1$: 在边缘服务器上执行
- $loc_i = D + E, D + E + 1, \dots, D + E + C - 1$: 在云服务器上执行

f_i 表示分配给任务 i 的计算资源， Δ_i 表示任务 i 的更新间隔。两者均为连续变量，在优化过程中根据资源上下限进行调整。

4.3.1 编码原理解释

这种编码方式有以下几个优势：

1. **直接映射到决策变量**: 编码中的 loc_i 直接对应于二进制决策变量 $x_{i,j}^d$ 、 $x_{i,j}^e$ 和 $x_{i,j}^c$, 表示任务的执行位置。通过一个整数变量 loc_i 可以替代多个二进制变量, 减少了问题的维度。
2. **简化约束处理**: 使用位置编码可以自动满足”每个任务只能分配到一个执行位置”的约束, 无需额外的约束处理机制。
3. **适应启发式算法**: 大多数启发式算法(如遗传算法、粒子群优化、灰狼优化等)都更适合处理连续或整数变量, 而不是大量二进制变量。这种编码将混合整数规划问题转换为更易优化的形式。
4. **降低搜索空间**: 通过使用整数表示位置, 而不是多个二进制变量的组合, 减少了搜索空间的大小, 提高了算法效率。

4.3.2 编码示例

假设一个系统包含 2 个设备 ($D = 2$)、3 个边缘服务器 ($E = 3$) 和 2 个云服务器 ($C = 2$), 以及 5 个任务需要处理。下面给出几个任务编码的示例:

- 任务 1 的编码: $S_1 = [0, 2.4]$
 - $loc_1 = 0$: 任务 1 在设备 0 上执行
 - $f_1 = 2.4$: 分配给任务 1 的 CPU 频率是 2.4GHz
- 任务 2 的编码: $S_2 = [3, 3.6]$
 - $loc_2 = 3$: 任务 2 在边缘服务器 1 上执行 (因为 $D = 2$, 所以 2 是第一个边缘服务器的索引, 3 是第二个)
 - $f_2 = 3.6$: 分配给任务 2 的 CPU 频率是 3.6GHz
- 任务 3 的编码: $S_3 = [6, 4.0]$
 - $loc_3 = 6$: 任务 3 在云服务器 1 上执行 (因为 $D + E = 5$, 所以 5 是第一个云服务器的索引, 6 是第二个)
 - $f_3 = 4.0$: 分配给任务 3 的 CPU 频率是 4.0GHz

如果考虑第二部分中的 AoI, 则示例如下:

- 任务 4 的编码: $S_4 = [1, 2.8, 0.5]$

- $loc_4 = 1$: 任务 4 在设备 1 上执行
- $f_4 = 2.8$: 分配给任务 4 的 CPU 频率是 2.8GHz
- $\Delta_4 = 0.5$: 任务 4 的更新间隔是 0.5 秒
- 任务 5 的编码: $S_5 = [4, 3.2, 0.8]$
 - $loc_5 = 4$: 任务 5 在边缘服务器 2 上执行
 - $f_5 = 3.2$: 分配给任务 5 的 CPU 频率是 3.2GHz
 - $\Delta_5 = 0.8$: 任务 5 的更新间隔是 0.8 秒

在启发式算法中, 这些编码形成了”解”或”个体”。算法会生成多个这样的解, 并通过迭代不断改进它们, 以寻找最优的任务卸载和资源分配方案。

5 基于教学优化的求解算法

5.1 TLBO 算法原理

基于教学的优化算法 (Teaching-Learning-Based Optimization, TLBO) 是一种模拟教学过程的元启发式优化算法, 由 Rao 等人于 2011 年提出。TLBO 算法模拟了一个班级的教学-学习过程, 其中解的种群被视为班级中的学生, 最优解被视为教师。TLBO 算法不需要任何特定的算法参数设置 (如变异率、交叉率等), 因此在参数调优方面具有优势。

TLBO 算法主要包括两个阶段: 教师阶段 (Teacher Phase) 和学习者阶段 (Learner Phase)。

5.1.1 教师阶段

在教师阶段, 学生通过接受教师的指导来改进自己的知识。教师试图将班级的平均水平提高自己的水平。对于第 i 次迭代, 教师阶段的更新公式如下:

$$X_{new,j} = X_{old,j} + r_j \cdot (X_{teacher,j} - T_F \cdot M_j) \quad (60)$$

$$M_j = \frac{1}{N} \sum_{k=1}^N X_{k,j} \quad (61)$$

其中:

- $X_{new,j}$ 是更新后的解在第 j 维的值
- $X_{old,j}$ 是当前解在第 j 维的值
- $X_{teacher,j}$ 是当前教师解 (即当前最优解) 在第 j 维的值
- M_j 是所有学生 在第 j 维的平均值
- r_j 是 $[0, 1]$ 范围内的随机数
- T_F 是教学因子, 通常随机取值为 1 或 2
- N 是种群大小 (学生数量)

如果新解优于旧解, 则接受新解; 否则, 保留旧解。

5.1.2 学习者阶段

在学习者阶段, 学生通过相互交流来提高知识水平。每个学生随机选择另一个学生进行学习, 如果另一个学生更好, 则向其学习; 否则, 远离它。更新公式如下:

对于学生 i 选择学生 $j \neq i$:

如果 $f(X_i) < f(X_j)$ (假设最小化问题):

$$X_{new,i} = X_{old,i} + r \cdot (X_i - X_j) \quad (62)$$

如果 $f(X_i) > f(X_j)$:

$$X_{new,i} = X_{old,i} + r \cdot (X_j - X_i) \quad (63)$$

其中 r 是 $[0, 1]$ 范围内的随机数。同样, 如果新解优于旧解, 则接受新解; 否则, 保留旧解。

5.2 TLBO 算法优势

TLBO 算法相比其他启发式优化算法具有以下优势:

1. 参数简单: 不需要设置特定的算法参数, 仅通过教学过程自然实现优化。

2. 收敛性好：通常可以较快地收敛到全局或近似全局最优解。
3. 实现简单：算法思想直观，易于编程实现。
4. 适应性强：适用于多种优化问题，包括连续、离散、多目标优化问题。

5.3 TLBO 算法流程

TLBO 算法的完整流程如下：

Algorithm 1 基于教学的优化算法 (TLBO)

```
1: 初始化参数: 种群大小  $N$ , 最大迭代次数  $MaxIter$ , 目标函数  $f(X)$ 
2: 随机初始化种群  $P = \{X_1, X_2, \dots, X_N\}$ 
3: 评估每个解的目标函数值  $f(X_i)$ 
4: for  $iter = 1$  to  $MaxIter$  do
5:   找出当前最优解  $X_{best}$  作为教师
6:   计算种群在各维度的平均值  $M$ 
7:   // 教师阶段
8:   for  $i = 1$  to  $N$  do
9:     随机生成教学因子  $T_F \in \{1, 2\}$ 
10:    生成随机向量  $r \in [0, 1]^D$ , 其中  $D$  是解的维度
11:     $X_{new,i} = X_i + r \cdot (X_{best} - T_F \cdot M)$ 
12:    评估  $X_{new,i}$  的目标函数值  $f(X_{new,i})$ 
13:    if  $f(X_{new,i}) < f(X_i)$  then
14:       $X_i = X_{new,i}$ 
15:    end if
16:  end for
17:  // 学习者阶段
18:  for  $i = 1$  to  $N$  do
19:    随机选择另一个学习者  $j \neq i$ 
20:    生成随机数  $r \in [0, 1]$ 
21:    if  $f(X_i) < f(X_j)$  then
22:       $X_{new,i} = X_i + r \cdot (X_i - X_j)$ 
23:    else
24:       $X_{new,i} = X_i + r \cdot (X_j - X_i)$ 
25:    end if
26:    评估  $X_{new,i}$  的目标函数值  $f(X_{new,i})$ 
27:    if  $f(X_{new,i}) < f(X_i)$  then
28:       $X_i = X_{new,i}$ 
29:    end if
30:  end for
31:  更新全局最优解  $X_{best}$ 
32: end for
33: return 全局最优解  $X_{best}$  及其目标函数值  $f(X_{best})$ 
```
